

Thread-safe printf Debugging (Part 2)

by Nathan Jones

Table of Contents

Introduction.....	1
Where did we leave off?	1
Preventing the second race: Disable interrupts.....	2
Preventing the first race	3
Disable interrupts everywhere.....	3
Use a mutex.....	4
Disable interrupts surgically.....	5
Use C/C++ atomics.....	6
Preventing the third race	7
“Couldn’t I just use a lock-free buffer?”	7
Wait, printf, not you too?!?	8
Are we not done yet?!?	9
Conclusion	9
Solutions.....	10
Race condition on xSemaphoreGiveFromISR().....	10

Introduction

Debugging via `printf` (or `Serial.print` or `snprintf`) can be an extremely useful tool for quickly seeing what your system is doing and to zero in on the parts of your system that are, or could soon be, causing errors. This tool is not without its downsides, however, and we can potentially run into trouble if we use certain naive implementations from inside ISRs (interrupt service routines) or between RTOS threads. In this article we’ll discuss what problems could potentially arise in these use cases and then do our best to make `printf/snprintf` **thread-safe** for use in ISRs and RTOS threads.

In the [last article](#), we saw that **buffering** our messages before they were sent out the UART/USB port helped reduce blocking time (copying to memory is far faster than waiting for serial characters to be sent out a hardware peripheral) but it came at the cost of several **race conditions** which need to be fixed in order for our code to still be **thread-safe**. We’ll look at those solutions here, along with solutions for other parts of our code that may not be thread-safe.

NOTE: As before, we are not going to address multi-core processors in this article. I’m going to assume that all code shown is running on a single-core processor only.

Where did we leave off?

At the end of the last article, we decided to use a simple, single buffer to meter out our messages to the UART port (Listing 1).

Listing 1: A simple, 256-byte buffer

```
#define MAX_MSG_LEN 256
char buffer[MAX_MSG_LEN+1] = {0};
volatile size_t nextIdx = 0, msgLen = 0;

size_t post(char * data, size_t len)
{
    // TODO: Handle data == NULL, len == 0
    size_t copied = 0;
    if(msgLen == 0) // Alt: if(!UART_TXE_IS_ENABLED())
    {
        msgLen = (len > MAX_MSG_LEN) ? MAX_MSG_LEN : len;
        memcpy(buffer, data, msgLen);
        nextIdx = 0;
        copied = msgLen;
        UART_TXE_ENABLE();
    }
    return copied;
}

void UART_TXE(void)
{
    UART_DR = buffer[nextIdx++];
    if(nextIdx == msgLen)
    {
        msgLen = 0;
        UART_TXE_DISABLE();
    }
}
```

Analysis of that code revealed three possible race conditions, which were:

1. Two threads/ISRs enter the `if(msgLen == 0)` block in `post()` at the same time
2. An ISR interrupts `UART_TXE_ISR` after `msgLen = 0` but before `UART_TXE_DISABLE()`
3. An optimizing compiler/out-of-order processor moves the `memcpy(buffer, data, msgLen)` in `post()` to be *after* `UART_TXE_ENABLE()`

Let's now discuss how to use our "thread-safe toolbox" to eliminate those race conditions.

Preventing the second race: Disable interrupts

Because *reasons*, I want to first discuss how to address the second race condition: the one where the UART TXE ISR could get preempted after `msgLen = 0`, which would cause the ISR to be permanently disabled.

To fix this, we need to find a way to prevent any other ISRs from running once the UART TXE ISR has determined that `nextIdx` equals `msgLen`. The only tool from our toolbox that works for us is to disable all interrupts around that code. The code snippet below shows how to do this for ARM Cortex-M devices by using CMSIS library functions.

Listing 2: A thread-safe UART_TXE_ISR

```
void UART_TXE(void)
```

```

{
    UART_DR = buffer[nextIdx++];
    __disable_irq();
    if(nextIdx == msgLen)
    {
        msgLen = 0;
        UART_TXE_DISABLE();
    }
    __enable_irq();
}

```

A mutex protecting the ISR status wouldn't work here because the UART ISR couldn't wait if another thread had that mutex (ISRs shouldn't block at all) and it couldn't simply skip the part where it turns itself off.

Preventing the first race

To prevent the first race condition (where two or more threads could be inside the main `if` block of `post()`), we need to ensure that once a thread has checked `msgLen`, nothing else can execute until that thread updates `msgLen`. We can do this in a few ways.

Disable interrupts everywhere

The easiest method is to disable interrupts throughout `post()`. The code below shows how to do this for ARM Cortex-M devices.

Listing 3: Disabling interrupts for a thread-safe post()

```

size_t post(char * data, size_t len)
{
    uint32_t primask_state = __get_PRIMASK();
    __disable_irq();
    ...
    __set_PRIMASK(primask_state);
    return copied;
}

```

Notice that we don't actually turn ISRs back on at the end of `post()`, we merely restore whatever interrupt status was active when the function was entered. This helps ensure that if interrupts were *off* at the top of `post()` (like they might be if the calling function were controlling another hardware peripheral when it called `post()`), they aren't accidentally turned back on at the end of the function.

(Why didn't we use this same code above in Listing 2? Because if the ISR is running then we already know that ISRs are enabled. We'd only ever have any problems if, after `UART_TXE_ISR` began, a higher-priority ISR ran that turned off interrupts and left them off, but that seems like bad practice to me, so I'll not account for it here. If you want to be overly cautious, the code in Listing 3 will work fine in place of what we have in Listing 2.)

Use a mutex

Alternatively, we can protect the entire function with a mutex. The code below shows how to do this for FreeRTOS (note that `postMutexInit()` needs to be called during system initialization in order for the mutex to be valid).

Listing 4: Acquiring a mutex for a thread-safe post()

```
SemaphoreHandle_t postMutex = NULL;
StaticSemaphore_t postMutexBuffer;

void postMutexInit(void)
{
    postMutex = xSemaphoreCreateMutexStatic( &postMutexBuffer );
}

size_t postFromTask(char * data, size_t len)
{
    size_t copied = 0;
    if( xSemaphoreTake(postMutex, 0)
    {
        ...
    }
    return copied;
}
```

We want to lock the buffer until the message is done being sent, so we'll wait to unlock the mutex until we're in the UART TXE ISR and the message has been sent.

Listing 5: Releasing the mutex in UART_TXE_ISR

```
void UART_TXE(void)
{
    signed BaseType_t yield = pdFALSE;
    UART_DR = buffer[nextIdx++];
    __disable_irq();
    if(nextIdx == msgLen)
    {
        msgLen = 0;
        xSemaphoreGiveFromISR(postMutex, &yield);
        UART_TXE_DISABLE();
    }
    __enable_irq();
    portYIELD_FROM_ISR(yield);
}
```

It's necessary to use `xSemaphoreGiveFromISR` (and to include `portYIELD_FROM_ISR` at the end) to both (1) not block and (2) alert the scheduler to any tasks that may have become unblocked by the UART TXE ISR releasing the mutex.

Challenge question!

If ISRs *weren't* disabled globally in the ISR above, we'd have *another* race condition based on the fact that `xSemaphoreGive()` happens before `UART_TXE_DISABLE()`. How would that race play out? (Solution [here](#).)

The fact that `postFromTask( )` simply returns if it can't immediately acquire the mutex should be key to allowing this function to be called from inside other ISRs, but the FreeRTOS documentation is weird about calling `xSemaphoreTake( )` from an ISR and instead recommends a more convoluted approach with the more aptly named `xSemaphoreTakeFromISR( )`. This will, unfortunately, require a separate function with a slightly altered function signature.

Listing 6: Acquiring a mutex for an ISR-safe `postFromIsr()`

```
size_t postFromIsr(char * data, size_t len, signed BaseType_t * yield)
{
    size_t copied = 0;
    if( xSemaphoreTakeFromISR(postMutex, yield) )
    {
        ...
    }
    return copied;
}
```

The calling ISR also needs to yield to the FreeRTOS scheduler if the `yield` argument (which is set by `xSemaphoreTakeFromISR`) is set to `pdTRUE`.

Listing 7: Correct usage of `postFromIsr()`

```
void thisIsr(void)
{
    ...
    char * msg = "Hello, is it me you're looking for?\n";
    signed BaseType_t yield == pdFALSE;
    size_t copied = postFromIsr(msg, strlen(msg), &yield);
    if(copied == 0) // Handle message not sent
    if(yield != pdFALSE) taskYIELD();
}
```

The code from the last two sections (Listings 3-7) solves the race condition and they each block for less time than if we'd disabled interrupts around a call to `printf`. This is definitely an improvement. But we can do better!

Disable interrupts surgically

Instead of disabling interrupts the entire time, we'll disable them around *just* the parts that can't be interrupted which, if you remember from our discussion above, was really just the two lines:

```
if(msgLen == 0){
    msgLen = (len > MAX_MSG_LEN) ? MAX_MSG_LEN : len;
```

As long as those two lines can complete without interruption or preemption, then we can eliminate our first race condition. Here's what that might look like if we were disabling interrupts for that portion:

Listing 8: Disabling interrupts surgically for a thread-safe `post()`

```
size_t post(char * data, size_t len)
{
    // TODO: Handle data == NULL, len == 0
    size_t copied = 0;
    uint32_t primask_state = __get_PRIMASK();
```

```

__disable_irq();
if(msgLen == 0) // Alt: if(!UART_TXE_IS_ENABLED())
{
    msgLen = (len > MAX_MSG_LEN) ? MAX_MSG_LEN : len;
    __set_PRIMASK(primask_state);
    memcpy(buffer, data, msgLen);
    nextIdx = 0;
    copied = msgLen;
    UART_TXE_ENABLE();
}
__set_PRIMASK(primask_state);
return copied;
}

```

In this case, we need to (potentially) turn interrupts back on in two locations, one each for the two branches of the `if` statement (taken or not taken).

The best part of this option is that we're not waiting on `memcpy` to finish before being able to turn interrupts back on again; we were able to move it outside the critical section.

Use C/C++ atomics

A third tool in our toolbox! Since C11/C++11, programmers have had access to atomic data types and atomic functions via `stdatomic.h` (C) or `cstdatomic` (C++). One such function that will be useful for us is `atomic_compare_exchange_strong`, which tests if a variable (say, `msgLen`) matches an expected value (0) and, if it does, sets it to a desired value (`minLen`), all completely atomically.

Listing 9: Using atomics for a thread-safe post()

```

#include <stdatomic.h>

size_t post(char * data, size_t len)
{
    size_t copied = 0;
    size_t expected = 0;
    size_t minLen = (len > MAX_MSG_LEN) ? MAX_MSG_LEN : len;
    if( atomic_compare_exchange_strong(&msgLen, &expected, minLen) )
    {
        memcpy(buffer, data, msgLen);
        nextIdx = 0;
        copied = msgLen;
        UART_TXE_ENABLE();
    }
    return copied;
}

```

This works best on processors that have atomic machine instructions such as compare-and-swap, load-link, and store-conditional. On processors that don't, `atomic_compare_exchange_strong` may simply compile into a pair of interrupt disable/enable instructions. Be careful, though! On Cortex-M0 processors, [GCC may silently convert this “atomic” function into a decidedly non-atomic series of machine instructions!](#)

Preventing the third race

To prevent the third race, the one where an optimizing compiler or out-of-order processor could move `UART_TXE_ENABLE( )` above the code that sets up the buffer, we need a way to let the compiler and/or processor know when instructions can't be moved around. The solution is to use a **barrier** (memory barrier, instruction barrier, or both). If we're enabling and disabling interrupts using [`\_\_disable\_irq\(\)`](#)/[`\_\_set\_primask\(\)`](#) (for global interrupt enable/disable) and [`\_\_NVIC\_EnableIRQ\(\)`](#)/[`\_\_NVIC\_DisableIRQ\(\)`](#) (for ISR-specific enable/disable, like `UART_TXE`) then the proper memory barriers are already in place. Take a look at the links above to see.

(In this specific case it would also have worked to have marked `buffer` as a `volatile char[ ]`, since the compiler is prevented from re-ordering volatile memory accesses around one another. However, making `buffer` a `volatile` isn't strictly necessary for the code to work correctly and adding `volatile` where it's not needed may actually hide race conditions that should be explicitly dealt with, as John Regehr notes in [“Nine ways to break your systems code using volatile”](#).)

Barriers are also critical for preventing **even more race conditions** that could occur in advanced processors (those with pipelines or data caches), stemming from the fact that some instructions may still get to execute after interrupts have been enabled or disabled, if they're already in the processor's instruction pipeline or if a cache line hasn't been written out to main memory yet.

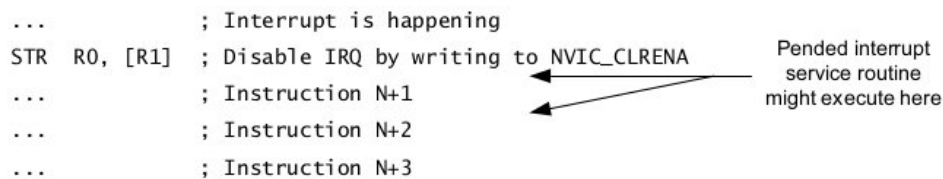


Figure 13 Implemented interrupt disabling delay in Cortex-M processors

From [ARM Cortex-M Programming Guide to Memory Barrier Instructions Application Note 321](#).

Correctly using memory barriers is at least an entire article in itself, so I'll say no more about it here. If you're interested in learning more about the proper use of memory barriers for ARM Cortex-M devices, refer to [ARM Cortex-M Programming Guide to Memory Barrier Instructions Application Note 321](#).

In short: **Use the CMSIS functions when enabling or disabling interrupts.**

“Couldn't I just use a lock-free buffer?”

A “lock-free” buffer is a buffer that can correctly synchronize memory accesses *without* using mutexes (i.e. locks), usually using atomic instructions in some very clever way to do so. The allure is nice: a lock-free buffer would mean multiple threads (ISRs, even!) could post messages to our buffer at the same time without blocking each other, no need for disabling ISRs or acquiring mutexes. It would be a complete “win-win”! And, theoretically, it's absolutely possible to write a lock-free buffer. But I'm going to **strongly discourage** you from attempting to do so or from using any “lock-free” code you find on the internet, unless you have an advanced degree in writing lock-free algorithms. Writing a

lock-free algorithm that *looks* like it will work is the easy part; it's *proving* that your algorithm is thread-safe that's massively difficult (and many that claim to be, aren't).



https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcSnNF_WyuBWUCMfE353ZCT-S3oOao7a8XAzmWQ&s

Writing a lock-free algorithm is like coming up with your own knot: sure, you can wrap a string around and through itself a few times, but more often than not when you pull on the ends it'll just fall apart.

Wait, printf, not you too!?

Although `post()` may be thread-safe now, that's only one function in our system. The functions `printf` and `sprintf` (among many others in the standard library) are *not* inherently thread-safe, possibly because they may modify static variables but mostly because they can call `malloc`, which itself is not thread-safe. Making these functions thread-safe requires:

- Creating a `__reent` struct for each thread (`struct __reent myReentrancyStruct = _REENT_INIT(myReentrancyStruct);` don't forget to `#include <reent.h>`) and setting the value of `_impure_ptr` to that struct upon each context switch (`_impure_ptr = &myReentrancyStruct`)
 - Thankfully many RTOSes will do this for you automatically; in FreeRTOS, just set `configUSE_NEWLIB_REENTRANT` to 1
- Using the `_r` version of the library function (e.g. `_sprintf_r`), [where available](#)
- Implementing `void __malloc_lock(struct _reent *ptr)` and `void __malloc_unlock(struct _reent *ptr)` (using, perhaps, a FreeRTOS mutex) to protect concurrent requests for heap memory.

NOTE: [Dave Nadler discovered](#) that both ST and NXP currently provide **inconsistent** FreeRTOS + newlib integration, such that any call to `malloc/free` in a FreeRTOS application (including any made from inside a vendor library) results in memory corruption. [Dave's fix](#) was to wrap `malloc/free` with the FreeRTOS memory API so that all dynamic memory requests are handled correctly and in the same place.

There are other `printf` libraries that claim to be thread-safe, but given how hard it is to write thread-safe code, I don't think I would trust any such claims until I'd done a personal audit of the code. The one exception, for me, might be the SEGGER RTT library (which I mentioned in [“Smaller printf Debugging”](#)). SEGGER is a highly reputable company and seems to have thought through thread-safety: the functions `SEGGER_RTT_LOCK()` and `SEGGER_RTT_UNLOCK()` can be seen throughout the code, typically at the top and bottom of each library function. The LOCK/UNLOCK functions are defined to simply turn off/on ISRs, but can be elided by #defining `SEGGER_RTT_LOCK / SEGGER_RTT_UNLOCK`, in which case the developer would provide their own implementations for `SEGGER_RTT_LOCK()` and `SEGGER_RTT_UNLOCK()`. I might still want to do a code audit, though...

Are we not done yet?!?

Nope, sorry, not even close. So far we've made `post()` and `printf/snprintf` thread-safe. In the three “Faster printf” articles we were using a double buffer inside `post()`, though, not a single buffer, and when we moved to tokenizing our debug messages we changed out `sprintf` for, alternately

- `mpack_writer_init/mpack_start_array/mpack_write_uint/mpack_finish_array` (from `mpack`),
- `mutate_value` (from `FlatBuffers`), or
- `EncodeCounter` (from `BitProto`).

Which of these functions are thread-safe? I don't know! (That's a lie; `EncodeCounter` is simple enough to inspect it and determine that it's thread-safe.)

Furthermore, which of the other dozens (hundreds? thousands??) of other functions in your application are truly thread-safe? AFAIK, the only way to tell is through a combination of thorough source code analysis, tooling to help detect data races, and lengthy stress testing. Good luck.

Conclusion

Writing thread-safe code is hard. Damn hard. But it's kind of a necessity if you choose to use ISRs or RTOS threads in your application and you don't want your application to randomly fail.

For making `printf` thread-safe, the simplest solution is to surround any call to `printf` with code that disables IRQs (selectively re-enabling them after) or acquires and releases a mutex. The downside to either option is that the rest of the program is blocked (some or all), possibly for a long time.

Posting to a buffer greatly shortens the blocking time; now we only need to disable IRQs/hold the mutex as long as it takes to copy a message into the buffer, not necessarily the entire time it would take to transmit. However, now we have to be more careful about identifying and preventing race conditions.

To identify race conditions, we need to look out for a few things in our code:

1. Code that accesses shared data, such as hardware peripherals

2. Code that is intended to be “atomic”, i.e. “indivisible”; in other words, a section of code that, once begun, *must* be finished before another thread should be allowed to enter the same section
3. Code that can’t happen until after a specific event
4. Code that has to happen immediately after a specific event

Once identified, we can use one or more of the three tools in our thread-safe toolbox to eliminate those races. Our three tools are:

1. Disable all interrupts before the critical section
2. Require that each thread acquire a mutex before entering a critical section
3. Use atomics to protect a critical section

(This is not necessarily an exhaustive list, but it’s good enough for today.)

Also, if we’re doing all this work to make `post ( )` thread-safe, we need to make sure that the rest of our program is thread-safe as well. Ensuring that `snprintf` (and any other C library function) is thread-safe requires a bit of work with `<reent.h>`, and you might need mutexes to protect calls to `malloc` anyway. And don’t forget to check if `mpack_write_uint`, `mutate_value`, and all of your other functions are thread-safe, too!

If you’ve made it this far, may God have mercy on your soul. I mean, thanks for reading and happy hacking!

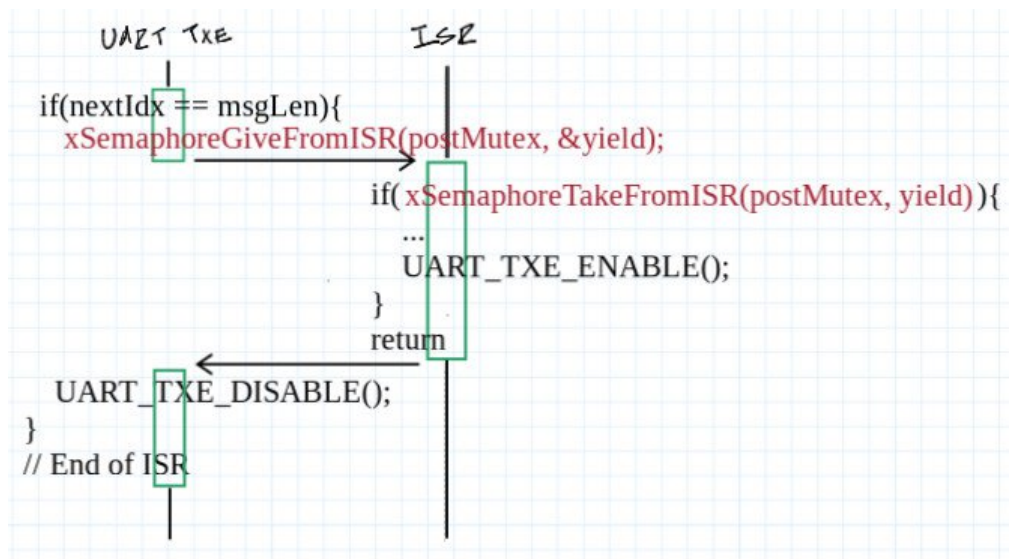
Solutions

Race condition on `xSemaphoreGiveFromISR( )`

Without the global interrupt disable, the UART TXE ISR would look like this:

```
void UART_TXE(void)
{
    signed BaseType_t yield = pdFALSE;
    UART_DR = buffer[nextIdx++];
    if(nextIdx == msgLen)
    {
        msgLen = 0;
        xSemaphoreGiveFromISR(postMutex, &yield);
        UART_TXE_DISABLE();
    }
    portYIELD_FROM_ISR(yield);
}
```

Since we’re using the mutex to control whether a thread is allowed to put a message into the buffer (i.e. to enter the `if` block in `post ( )`), releasing the mutex in the ISR, prior to disabling itself, is a race that plays out just like the second race in our list, just with the mutex instead of `msgLen`:



1. UART_TXE runs, determines that the message it's sending is done, and enters the `if(nextIdx == msgLen)` block. It sets ~~msgLen to 0~~ unlocks the mutex.
2. At that exact moment, a higher-priority ISR runs that wants to send out a message. It sees that ~~msgLen is 0~~ the mutex is unlocked, so it copies its contents into the buffer and enables the UART_TXE ISR.
3. The first instance of UART_TXE is returned to, which *disables the UART_TXE ISR*. No further messages can be sent out because, improbably, ~~msgLen is non-zero~~ the mutex is locked but the ISR has been disabled.